In [6]:

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"   # silence INFO/WARN from TF C++ backend
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "1"  # enable CPU oneDNN optimizations

# ========================
# Your Imports (plus stdlib/types)
# ========================
from __future__ import annotations

import warnings
from dataclasses import dataclass
from typing import List, Tuple, Dict, Optional

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, ExtraTreesRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.cluster import KMeans
from sklearn.impute import SimpleImputer
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

from xgboost import XGBRegressor


import holidays
```

```
import joblib
from tabulate import tabulate

# TensorFlow/Keras (after env vars above)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks, optimizers, models
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"   # set this before importing TF
import tensorflow as tf
import os
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "1"  # set before importing TF
import tensorflow as tf
tf.config.threading.set_intra_op_parallelism_threads(0)  # let TF decide (or set N)
tf.config.threading.set_inter_op_parallelism_threads(0)
```

In [7]:

```
def Data_Index(df):
    df['time'] = pd.to_datetime(df['time'], errors='coerce') # parse with offs
    df = df.set_index('time')
    return df
def Data_visualization(df):

    import matplotlib.pyplot as plt
    plt.figure(figsize=(12,6))
    plt.title("Customer Count Over time")
    plt.plot(df.index,df["customers"], linestyle ="-")
    plt.xlabel("Year")
    plt.ylabel("Customer")
    plt.grid(True)
    plt.show()
##############################################################################
# Feature Formation
##############################################################################
def Make_Baseline_features(df):
    df["year"] = df.index.year
    df["month"] = df.index.month
    df["day"] = df.index.day
    df["dayofweek"] = df.index.dayofweek
```

```
    df["quarter"] = df.index.quarter
    return df
def Make_Other_feature(df):
    df = df.copy()
    df["previous_day_customers"] = df["customers"].shift(1).fillna(0)
    return df

def Rolling_means(df):
    df = df.copy()

    shifted_customer = df["customers"].shift(1)
    df["rolling_means"] = shifted_customer.rolling(window =5, min_periods = 1).mean()
    df["rolling_means"] = df["rolling_means"].fillna(df["rolling_means"].mean())
    return df

def add_exponential_smoothing(df):
    df = df.copy()

    shifted_customer = df["customers"].shift(1)
    df["exponential_smoothing"] = shifted_customer.ewm(span =5 ,adjust= False ).mean()
    df["exponential_smoothing"] = df["exponential_smoothing"].fillna(df["exponential_smoothing"].mean())
    return df
def add_autocorrelation_feature(df: pd.DataFrame, window: int = 14, lag: int = 1) -> pd.DataFrame:
    df = df.copy()
    s = df["customers"].shift(1)

    roll_ac = s.rolling(window=14, min_periods=max(lag + 1, 3)).apply(lambda x: x.autocorr(lag=1), raw=False)
    df["autocorelation"] = roll_ac.fillna(0)

    return df
def Holiday_feature(df):

    fi_holidays = holidays.Finland(years = range(2019, 2025))
    holiday_idx = pd.DatetimeIndex(pd.to_datetime(list(fi_holidays.keys())))
    d = df.index.normalize()
    df["is_holiday"] = d.isin(holiday_idx).astype(int)
    df["day_before_holiday"] = (d + pd.Timedelta(days =1 )).isin(holiday_idx).astype(int)
    df["day_after_holiday"] = (d - pd.Timedelta(days =1 )).isin(holiday_idx).astype(int)
    return df
```

```
##########################################################################################################
df = pd.read_csv("Datasets/customers_data.csv")
df = Data_Index(df)
#Data_visualization(df)
df = Make_Baseline_features(df)
df = Make_Other_feature(df)
df = Rolling_means(df)
df = add_exponential_smoothing(df)
df = add_autocorrelation_feature(df)
df = Holiday_feature(df)
##########################################################################################################

df.columns
```

Out[7]:
```
Index(['customers', 'year', 'month', 'day', 'dayofweek', 'quarter',
       'previous_day_customers', 'rolling_means', 'exponential_smoothing',
       'autocorelation', 'is_holiday', 'day_before_holiday',
       'day_after_holiday'],
      dtype='object')
```

In [8]:
```
def Train_test_split(target, features):
    X = df[features]
    y  = df[target]
    X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.8, shuffle = False)
    return X,y, X_train, X_test, y_train, y_test

def Evaluate_model_performance(y_test,y_pred):

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test,y_pred)
    r2_score = (y_test, y_pred)
    print(f"MSE:{mse:.2f}")
    print(f"RMSE:{rmse:.2f}")
    print(f"MAE:{mae:.2f}")
    #print(f"R2:{r2_score:}")

# Model
def XGBoost_Model():
    model = XGBRegressor (
        n_estimators = 300,
```

```
        learning_rate = 0.1,
        max_depth = 5,
        subsample = 0.8,
        colsample_bytree =0.8,
        random_state = 42
    )
    return model
# === Evaluate ===

#############################################################################################################
df = df.dropna()
target = "customers"
features = [col for col in df.columns if col != target ]

model = XGBoost_Model()
X,y, X_train, X_test, y_train, y_test  = Train_test_split(target, features)
model.fit(X_train, y_train)
# === Predict ===
y_pred = model.predict(X_test)

Evaluate_model_performance(y_test,y_pred)
```

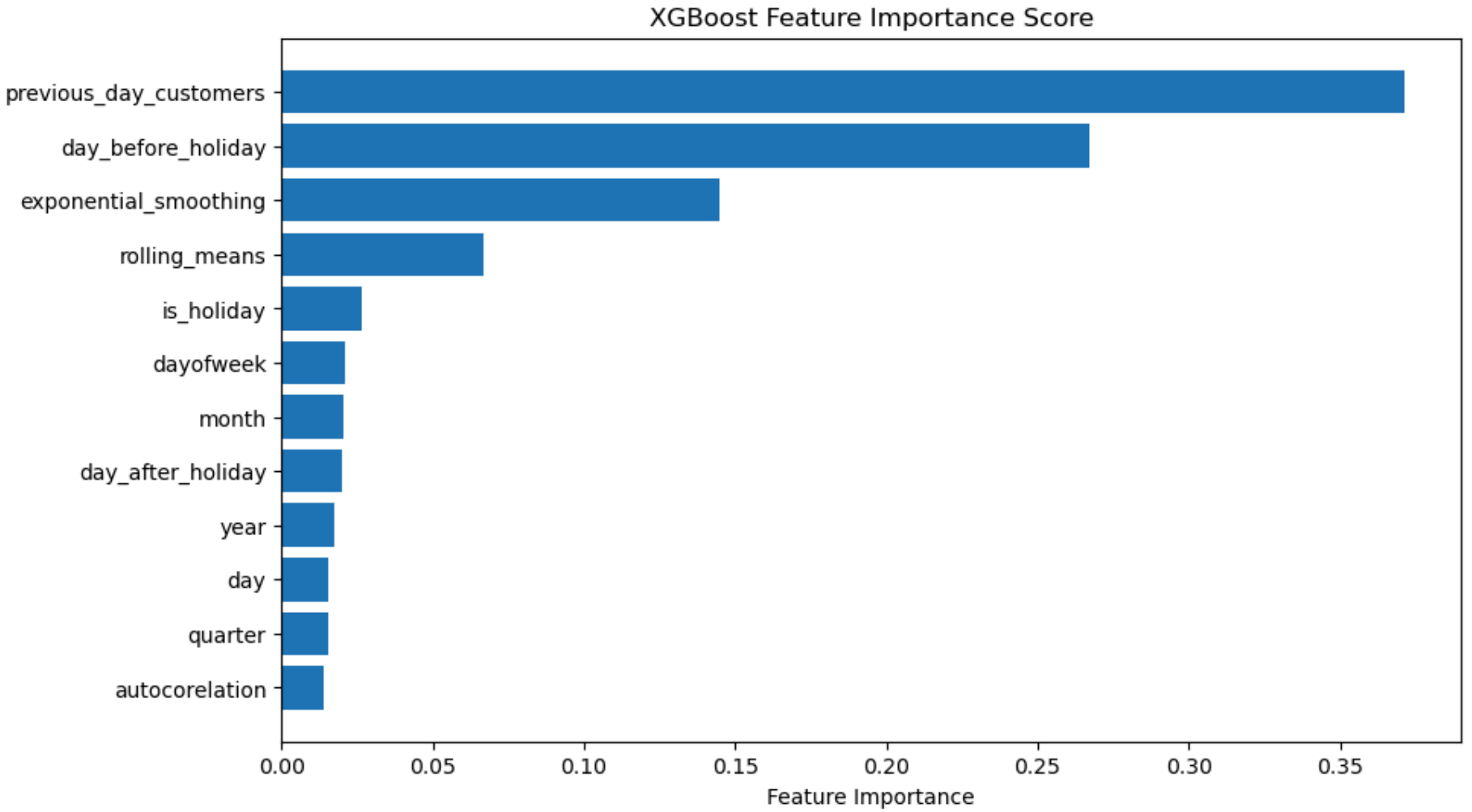
```
MSE:14640.44
RMSE:121.00
MAE:97.69
```

In [11]:

```
def XGBoost_Feature_importance(X):
    importance = model.feature_importances_
    feature_importance = pd.DataFrame({"feature": X.columns, "importance": importance})
    feature_importance = feature_importance.sort_values(by = "importance", ascending = False )
    return    feature_importance
def Visualize_XGBoost_Feature_importance(feature_importance):
    plt.figure(figsize=(10,6))
    plt.barh(feature_importance["feature"], feature_importance["importance"])
    plt.gca().invert_yaxis()
    plt.title("XGBoost Feature Importance Score")
    plt.xlabel("Feature Importance")
    plt.show()

#############################################################################################################
```

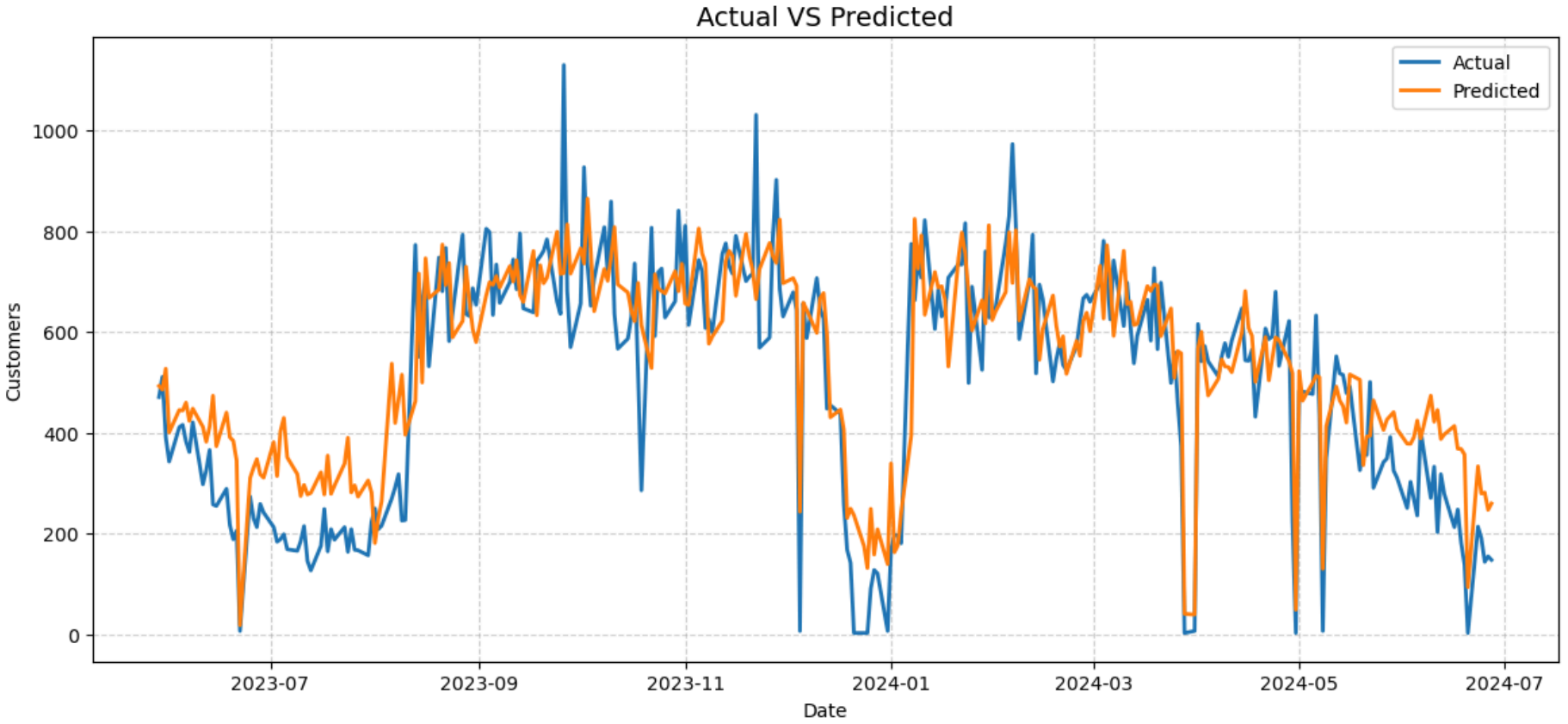
```
feature_importance = XGBoost_Feature_importance(X)
Visualize_XGBoost_Feature_importance(feature_importance)
```



In [ ]:

In [12]:
```
def Actual_vs_Predicted_Visualize():
    results = y_test.to_frame(name = "actual").copy()
    results["predicted"] = y_pred
    results.index = X_test.index
```

```
    return results
def Visulize_Actual_vs_Predicted_Visualize(results):
    plt.figure(figsize =(14,6))
    plt.plot(results.index, results["actual"], label ="Actual", linewidth =2)
    plt.plot(results.index, results["predicted"], label ="Predicted", linewidth =2)
    plt.title("Actual VS Predicted", fontsize =14)
    plt.xlabel("Date")
    plt.ylabel("Customers")
    plt.legend()
    plt.grid(True, linestyle ="--", alpha = 0.6)
    plt.show()
#################################################################################
results = Actual_vs_Predicted_Visualize()
Visulize_Actual_vs_Predicted_Visualize(results)
```



Part 3

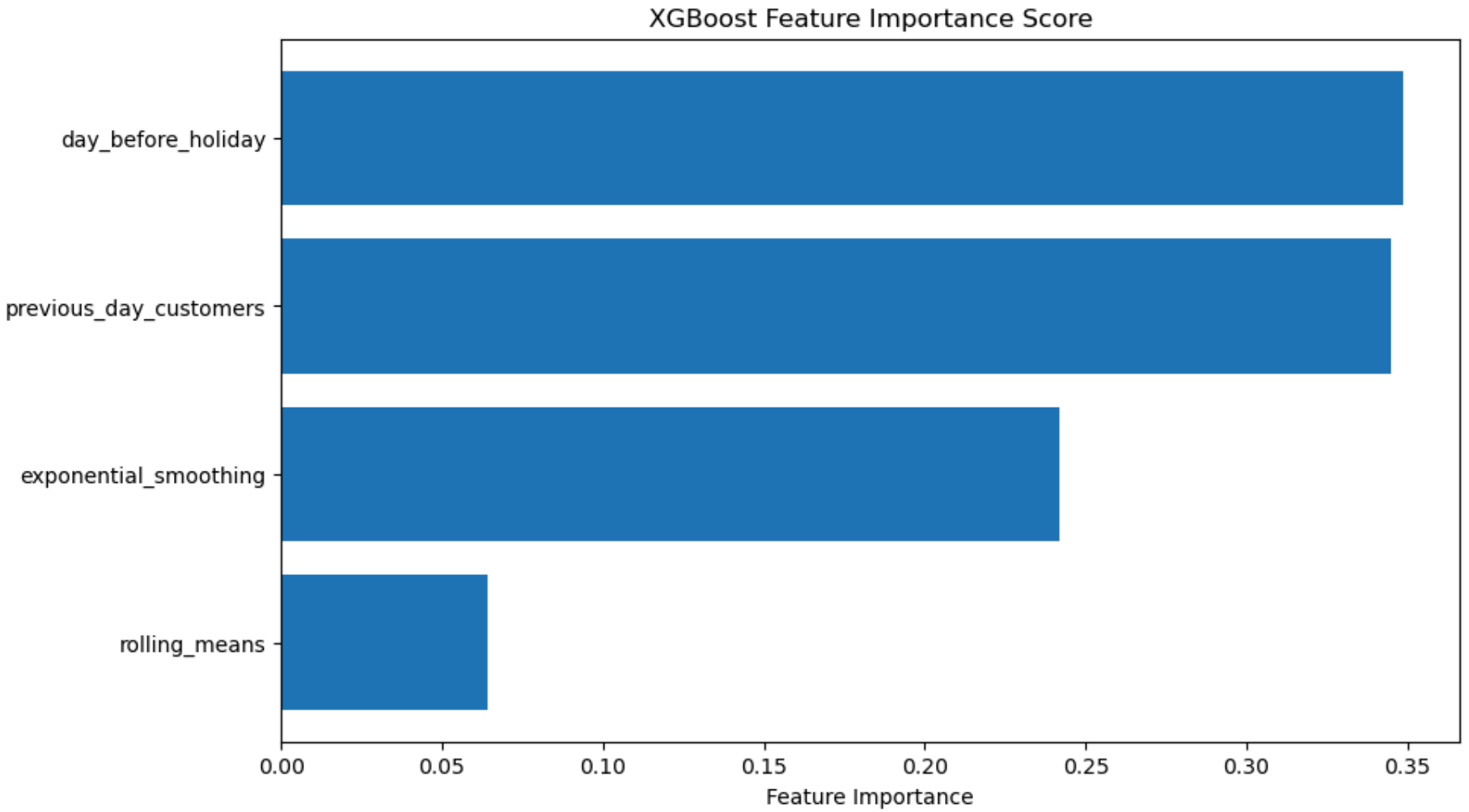
In [84]:
```
df = df.dropna()
#df = df[df["customers"] > 50]
target = "customers"
features = ["previous_day_customers","day_before_holiday", "exponential_smoothing","rolling_means"]

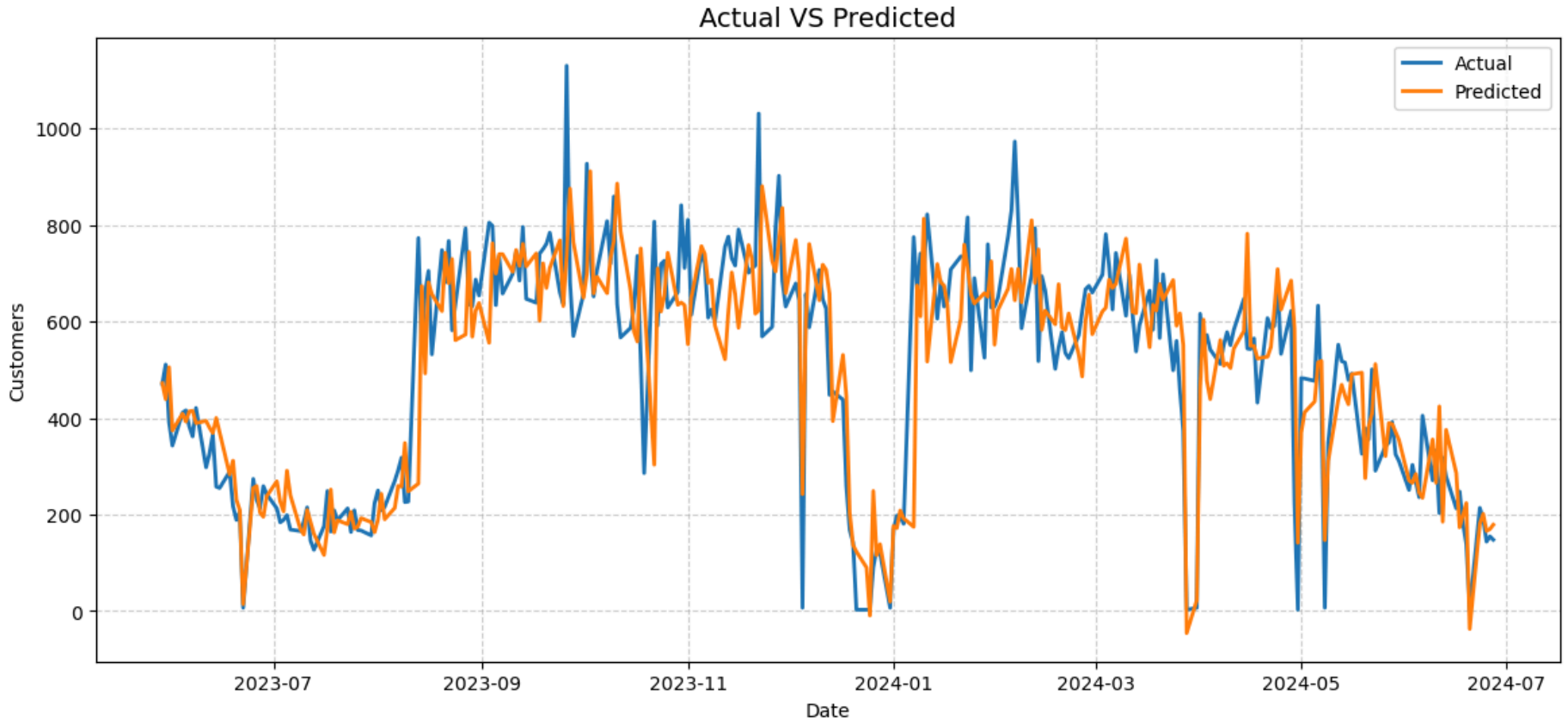
model = XGBoost_Model()
X,y, X_train, X_test, y_train, y_test  = Train_test_split(target, features)
model.fit(X_train, y_train)
# === Predict ===
y_pred = model.predict(X_test)

Evaluate_model_performance(y_test,y_pred)

#############################################################################################
feature_importance = XGBoost_Feature_importance(X)
Visualize_XGBoost_Feature_importance(feature_importance)
#############################################################################################
results = Actual_vs_Predicted_Visualize()
Visulize_Actual_vs_Predicted_Visualize(results)
```

```
MSE:15377.82
RMSE:124.01
MAE:88.03
```


XGBoost Feature Importance Score
day_before_holiday
previous_day_customers
exponential_smoothing
rolling_means
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Feature Importance



Random Forest Model

In [ ]:
```
Builds many trees independently and in parallel.
Averages predictions to reduce variance, making it less prone to overfitting.
Robust to noise and outliers; simpler to tune for a good baseline.
Requires extensive feature engineering (lags, seasonality, holidays) as it doesn't inherently understand time.

XGBoost (Boosting Ensemble)

Builds trees sequentially, each correcting errors of the previous ones.
Often achieves higher accuracy by focusing on difficult cases.
```

```
More prone to overfitting if not carefully tuned; requires more hyperparameter management.
Also requires extensive feature engineering for time series data.
```

In [21]:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

def random_forest_model():
    return RandomForestRegressor(
        n_estimators=500,        # plural
        max_depth=None,
        min_samples_split=2,     # plural + correct name
        min_samples_leaf=1,      # plural + correct name
        max_features="sqrt",     # plural
        bootstrap=True,
        n_jobs=-1,
        random_state=42
    )

def evaluate_model_performance(y_true, y_pred):
    mae  = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2   = r2_score(y_true, y_pred)
    print(f"MAE:  {mae:,.4f}")
    print(f"RMSE: {rmse:,.4f}")
    print(f"R²:   {r2:,.4f}")

# --- Prep ---
df = df.copy().dropna()
target = "customers"
features = [c for c in df.columns if c != target]

################################################################################

X = df[features]
y = df[target]

# --- Split ---
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# --- Train ---
model = random_forest_model()
model.fit(X_train, y_train)

# --- Predict & Evaluate ---
y_pred = model.predict(X_test)
evaluate_model_performance(y_test, y_pred)
```

```
MAE:  65.1230
RMSE: 103.9658
R²:   0.7780
```

Linear Regression

In [15]:

In [22]:
```
from sklearn.linear_model import LinearRegression
def linear_regression_forecast():
    lr_model = LinearRegression()
    return lr_model
```

In [23]:
```
df = df.dropna()
target = "customers"
features = [col for col in df.columns if col != target ]

model = linear_regression_forecast()
X,y, X_train, X_test, y_train, y_test  = Train_test_split(target, features)
model.fit(X_train, y_train)
# === Predict ===
y_pred = model.predict(X_test)

Evaluate_model_performance(y_test,y_pred)
```

```
MSE:13779.38
RMSE:117.39
MAE:83.72
```

In [24]:
```python
df = df[df["customers"] > 50]
df = df.dropna()

target = "customers"
features = ["previous_day_customers","day_before_holiday", "exponential_smoothing","rolling_means"]

model = linear_regression_forecast()
X,y, X_train, X_test, y_train, y_test  = Train_test_split(target, features)
model.fit(X_train, y_train)
# === Predict ===
y_pred = model.predict(X_test)

Evaluate_model_performance(y_test,y_pred)
```

```
MSE:15036.66
RMSE:122.62
MAE:87.53
```

In [25]:
```python
def split_data(df):
    train, test = train_test_split(df, test_size=0.2, shuffle=False)
    return train, test

# Function to calculate MSE, MAE, and RMSE
def calculate_metrics(predictions, actual):
    mse = mean_squared_error(actual, predictions)
    mae = mean_absolute_error(actual, predictions)
    rmse = np.sqrt(mse)
    return mse, mae, rmse

def lstm_forecast(train, test):
    data_train = train['customers'].values.reshape(-1, 1)
    data_test = test['customers'].values.reshape(-1, 1)

    scaler = MinMaxScaler()
    data_train_normalized = scaler.fit_transform(data_train)
    data_test_normalized = scaler.transform(data_test)
```

```
    time_steps = 3
    X_train, y_train = create_dataset(data_train_normalized, time_steps)
    X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))

    lstm_model = Sequential()
    lstm_model.add(LSTM(units=50, input_shape=(X_train.shape[1], 1)))
    lstm_model.add(Dense(1))
    lstm_model.compile(optimizer='adam', loss='mean_squared_error')
    lstm_model.fit(X_train, y_train, epochs=50, batch_size=1, verbose=0)

    X_test, y_test = create_dataset(data_test_normalized, time_steps)
    X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

    lstm_predictions = lstm_model.predict(X_test)
    lstm_predictions = scaler.inverse_transform(lstm_predictions)

    lstm_rmse = np.sqrt(mean_squared_error(data_test[time_steps:], lstm_predictions))
    lstm_mse = mean_squared_error(data_test[time_steps:], lstm_predictions)
    lstm_mae = mean_absolute_error(data_test[time_steps:], lstm_predictions)

    results = {
        'LSTM MSE': lstm_mse,
        'LSTM MAE': lstm_mae,
        'LSTM RMSE': lstm_rmse,
        'Predictions': lstm_predictions.flatten()
    }

    return results

def lstm_forecast(train, test):
    data_train = train['customers'].values.reshape(-1, 1)
    data_test = test['customers'].values.reshape(-1, 1)

    scaler = MinMaxScaler()
    data_train_normalized = scaler.fit_transform(data_train)
    data_test_normalized = scaler.transform(data_test)

    time_steps = 3
    X_train, y_train = create_dataset(data_train_normalized, time_steps)
    X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

```
    lstm_model = Sequential()
    lstm_model.add(LSTM(units=50, input_shape=(X_train.shape[1], 1)))
    lstm_model.add(Dense(1))
    lstm_model.compile(optimizer='adam', loss='mean_squared_error')
    lstm_model.fit(X_train, y_train, epochs=5, batch_size=1, verbose=0)

    X_test, y_test = create_dataset(data_test_normalized, time_steps)
    X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

    lstm_predictions = lstm_model.predict(X_test)
    lstm_predictions = scaler.inverse_transform(lstm_predictions)

    lstm_rmse = np.sqrt(mean_squared_error(data_test[time_steps:], lstm_predictions))
    lstm_mse = mean_squared_error(data_test[time_steps:], lstm_predictions)
    lstm_mae = mean_absolute_error(data_test[time_steps:], lstm_predictions)

    results = {
        'LSTM MSE': lstm_mse,
        'LSTM MAE': lstm_mae,
        'LSTM RMSE': lstm_rmse,
        'Predictions': lstm_predictions.flatten()
    }

    return results


# Function to create dataset for LSTM
def create_dataset(dataset, time_steps=1):
    X, y = [], []
    for i in range(len(dataset) - time_steps):
        a = dataset[i:(i + time_steps), 0]
        X.append(a)
        y.append(dataset[i + time_steps, 0])
    return np.array(X), np.array(y)

# Function to get RMSE of each model
def Get_Model_RMSE(df, feature_list):
    train, test = split_data(df)
    results_lstm = lstm_forecast(train, test)

    lstm_rmse = results_lstm['LSTM RMSE']
```

```
    lstm_mse = results_lstm['LSTM MSE']
    lstm_mae = results_lstm['LSTM MAE']
    print(lstm_rmse)
    print (lstm_mae)
    print ( lstm_mse)


df = df.dropna()
target = "customers"
features = [col for col in df.columns if col != target ]

Get_Model_RMSE(df, features)
```

```
2025-10-18 21:25:03.329013: E external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capable device is detected
/projappl/project_2014453/lib/python3.10/site-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
118.90423293536384
82.94377883535917
14138.216609947262
```

LSTM MODEL

In [26]:
```
def create_dataset(dataset, time_steps = 1):
    X, y = [], []
    for i in range(len(dataset)- time_steps):
        a = dataset[i:(i + time_steps), 0]
        X.append(a)
        y.append(dataset[i + time_steps,0])
    return np.array(X), np.array(y)
```

In [27]:
```
df = df.dropna()
target = "customers"
features = [col for col in df.columns if col != target ]
```

```
train, test = train_test_split(df, test_size=0.2, shuffle=False)
data_train = train["customers"].values.reshape(-1, 1)
data_test = test["customers"].values.reshape(-1, 1)

scaler = MinMaxScaler()
data_train_normalized = scaler.fit_transform(data_train)
data_test_normalized = scaler.transform(data_test)
time_steps = 3
X_train, y_train = create_dataset(data_train_normalized, time_steps)
X_train = np.reshape(X_train,(X_train.shape[0], X_train.shape[1],1))
```

In [ ]:
```
lstm_rmse = np.sqrt(mean_squared_error(data_test[time_steps:], lstm_predictions))          lstm_mse = mean_squared_error(data_
lstm_predictions = lstm_model.predict(X_test)    lstm_predictions = scaler.inverse_transform(lstm_predictions)
```

In [ ]:
```
lstm_model = Sequential()
lstm_model.add(Dense(1))
lstm_model.compile(optimizer='adam', loss='mean_squared_error')
lstm_model.fit(X_train, y_train, epochs = 50, batch_size = 1, verbose =0)
lstm_model = Sequential()
lstm_model.add(LSTM(units=50, input_shape=(X_train.shape[1], 1)))
lstm_model.add(Dense(1))
lstm_model.compile(optimizer='adam', loss='mean_squared_error')
lstm_model.fit(X_train, y_train, epochs=50, batch_size=1, verbose=0)
```

In [38]:
```
X_test, y_test = create_dataset(data_test_normalized, time_steps)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
lstm_predictions = lstm_model.predict(X_test)
lstm_predictions = scaler.inverse_transform(lstm_predictions)
lstm_rmse = np.sqrt(mean_squared_error(data_test[time_steps:], lstm_predictions))
lstm_mse = mean_squared_error( data_test[time_steps:],lstm_predictions)
lstm_mae = mean_absolute_error( data_test[time_steps:],lstm_predictions)
print(lstm_rmse)
print(lstm_mse)
print(lstm_mae)
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
115.61966609077288
13367.907186941817
80.87233302632316
```

Conclude

Feature Engineering

External features : Weather data, Elevator data, Menu food data, price for dishes

Apply outliers OR Anomalies detection